SOUTH LAKE HOME EDUCATORS



Lesson 8

Core Mechanics II: Under Pressure

Semester 1, Week 8 - Lose Condition + Lives System

At a Glance

Lesson 8 objectives & materials

By the end of today, students will:

- Reuse an existing code path to drive new, independent state
- Implement a lives counter and a lose condition
- Keep the win-counter and the lives-counter fully independent
- Explain why we're deliberately not building Game Over yet

What We're Using Today

- Demo laptop with all 8 engines verified
- Unity, for the live lecture demo
- No new assets this week - logic-only lesson
- No engine-specific bugfix this week, unlike Lesson 7
- Your own [YourName]Game project from Lesson 7 - not a new one

IGDA framework tie-in: Programming - win/lose conditions, event-driven programming (intermediate); Production - deliberate feature sequencing.

Today's Agenda

0:00 - 0:05	Welcome back, Lesson 7 recap
0:05 - 0:15	Reversing the Question: What Happens When THEY Reach YOU?
0:15 - 0:35	Reusing What You Built: The Escape Path Gets Stakes
0:35 - 0:50	Two Counters, Never Crossed
0:50 - 0:55	Why We're Not Building Game Over (Yet)
0:55 - 1:00	Introduce lab exercise, hand out engine-specific handouts
LAB TIME	***— Under Pressure —***
1:00 - 1:50	Add a lives system and lose condition to your project
1:50 - 2:00	Exit check / quick share-out



Welcome Back

Lesson 7 recap - your enemies started moving.

5 min

Welcome Back

5 min

Quick recap of Lesson 7:

- Continuous movement - enemies advance toward the player, every frame
- Escape cleanup - an enemy reaching the bottom unshot is simply removed, no consequence
- Today: that "no consequence" changes. An escape starts to matter.

Open your own project from last week today, not a new one - and no new assets needed for the core exercise, this is a logic-only lesson. Level 3 sound files are provided in this week's Assets folder.



Reversing the Question

What happens when an enemy reaches YOU?

10 min

Win AND Lose

10 min

Every lesson so far has asked: what happens when YOUR bullet reaches an enemy? Today's question is the opposite: what happens when an ENEMY reaches you?



Straight from the capstone's design document: Lose Condition - an enemy reaches the player's row, or the player runs out of lives. It's been on paper since Lesson 1 - today we build it.



Discussion: why does a game need both a win condition AND a lose condition? (Stakes, tension, replayability - a game you can't lose isn't very interesting to keep playing.)



Reusing What You Built

The escape path from Lesson 7 gets real stakes.

20 min

The Escape Path Gets Stakes

20 min

You already wrote the code that detects an enemy reaching the bottom - you built it last week for cleanup. Today we don't write new detection. We just give it a consequence.

Unity demo: Enemy.cs's existing escape branch gains one new line - a call to GameManager.RegisterEscape() - right before the Destroy(gameObject) call that was already there.

That small change isn't an accident - it's the payoff of building things in clean, separate pieces.

Discussion: when has good organization made new work easier for you, in this project or elsewhere?



Two Counters, Never Crossed

Kills and lives stay completely independent.

15 min

Two Counters, Never Crossed

15 min

Enemies Remaining

Changes only when you shoot something. Drives the WIN condition.

Lives

Changes only when something gets past you. Drives the LOSE condition.

These two numbers should NEVER affect each other. Today, test both directions: shoot one (lives shouldn't move) and let one escape (kill counter shouldn't move).



Why we're not building Game Over yet: when lives hits zero, "Game Over" appears - and the game just keeps running. Actually freezing everything and offering a restart button is real work, enough to get its own lesson (Lesson 10). Get the message right first; build the proper response later.



Today's Exercise & Lab

Under Pressure - a lives system and lose condition.

5 min

Under Pressure

5 min

An enemy reaching the bottom now costs a life. Lose all 3, and "Game Over" appears - built on top of your existing project.

Level 1 - Minimal

An escaped enemy costs a life (start at 3); "Game Over" appears at zero.

MVP - a complete finish for today.

Level 2 - Complete

Shooting never costs a life; escaping never counts as a kill.

Everything works end to end.

Level 3 - Refined

A live lives readout, a lose sound, and/or tighter code separation.

Ready to share.



Test both directions today: shoot an enemy (lives shouldn't move), then let one escape (your kill counter shouldn't move). Your handout's Level 2 section walks through exactly how to check this.

Lab Time Notes



- Most important check: shoot an enemy (lives should NOT change) and separately let one escape (the kill counter should NOT change). Check both directions.
- Most likely mistake: the lives decrement added to the wrong event/branch - easy to make quickly, easy to miss without deliberately testing both directions.
- No engine needs a structural bugfix this week, unlike Lesson 7 - circulate normally, no cluster needs extra attention by default.
- A student who lets every enemy through will hit Game Over quickly - a good built-in test case, not a bug, worth pointing out if they're confused.



Now There Are Stakes!

A lives system and a real lose condition, built almost entirely by reusing last week's work.

Next week: score tracking and a basic on-screen UI - showing your stats properly for the first time.

Try a Level 3 idea at home if you'd like - a live lives readout is a satisfying one to add.